



ಶರಣಬಸವ
Sharnbasva
ವಿಶ್ವವಿದ್ಯಾಲಯ
University
ಕಲಬುರಗಿ-585 103
ಕರ್ನಾಟಕ-ಭಾರತ
ಸ್ಥಾಪನೆ : 2017
Kalaburagi-585 103
Karnataka-India
Estd.: 2017




Faculty of: Engineering and Technology	Department: CSE/CSD Program: B.Tech	AY: 2026-27

INTERNAL ASSESSMENT (I.A.) TEST: I/II

SCHEME OF EVALUATION

Course Title: Operating Systems	Course Code: 25CS304
Semester: III Sem	Section: -
Course Teacher: Sharanabasappa Noola	Contact Number: 9035071855

Q.No.	Answer description	Marks Allotted
1 a)	MODULE - 1 Types of System calls & their operations are i) Process control — (1M) - A running program needs to be able to halt its execution either normally (end()) or abnormally (abort()). - load and execute a program - create process, terminate process - get process attributes, set process attributes Examples - fork(), exit(), wait(), exec(), getpid(). 2) File management — (1M) - we are able to create() and delete() files, - once created, we need to open() it and use it. - we may also read(), write(), or reposition() - finally, we need to close() the file, indicating that we are no longer using it. Examples open(), read(), write(), close()	6 M

Q.No.	Answer description	Marks Allotted

3) Device management — (1M)

- A process may need several resources to execute like main memory, disk drives, access to files & so on.
- If the resources are available, they can be granted, and control can be granted to the user process.
- A system with multiple users may require us to first request() a device,
- After we are finished with the device, we release() it.

Examples

ioctl(), read(), write()

4) Information maintenance — (1M)

* Most systems have a system call to return the current time() and date()
* other syster calls may return information about the system, such as number of current users, the version number of OS, the free space & so on.

Example

getpid(), alarm(), sleep(), time(), uname()

5) communication — (1M)

* These system calls are used to exchange information between process, either on the same computer or across networked process
* Here we can create and delete communication connections
* We can send and receive messages

Examples:- Pipe(), send(), recv(),

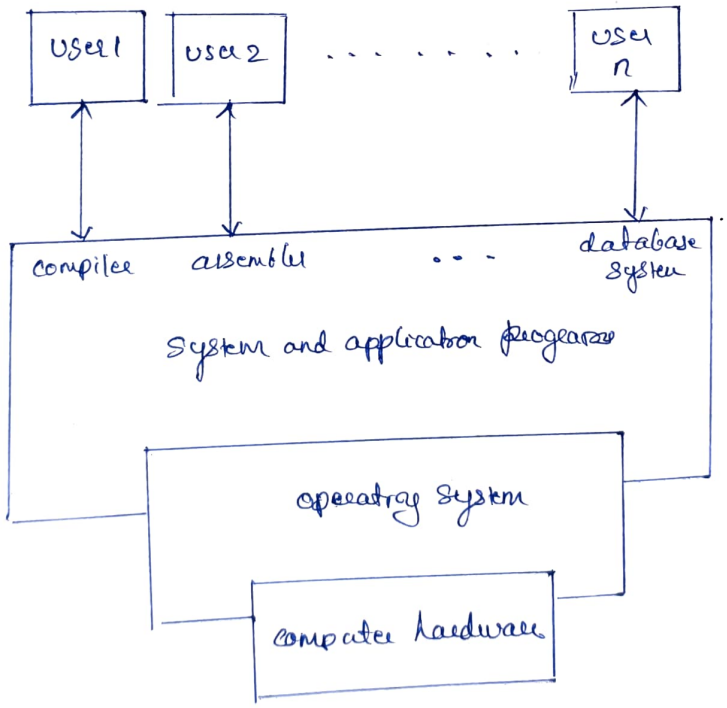
Q.No.	Answer description	Marks Allotted

6) Protection — (1 M)

* Protection provides a mechanism for controlling access to the resources provided by a computer system.
* Typically, system calls providing protection include set permission() and get permission(), which manipulate the permission settings of resources such as files and disks

Examples
chmod(), umask(), chown().

b)



6 M

-2 marks

Q.No.	Answer description	Marks Allotted

* The hardware - CPU, Memory, I/O devices — 1m
provides the basic computing resources for the system.

interaction with OS: The OS manages hardware resources and controls their allocation to different programs

* The operating system:- It is a system software — 1m
that acts as an interface between users, application programs, and computer hardware.
* It receives requests from application programs and coordinates with hardware to execute those requests.

- Application programs :- such as word processors, — 1m
spreadsheets, compilers, & web browsers - define the way in which these resources are used to solve user's computing problems.

interaction :- Applications request operating system service through system calls to access files, memory, processors and I/O devices

- Users :- users are individual or other systems — 1m
that interact with the computer to perform various tasks.

interaction :- Users interact with OS through a command-line interface or GUI to execute applications & perform operations.

Q.No.	Answer description	Marks Allotted

— OR —

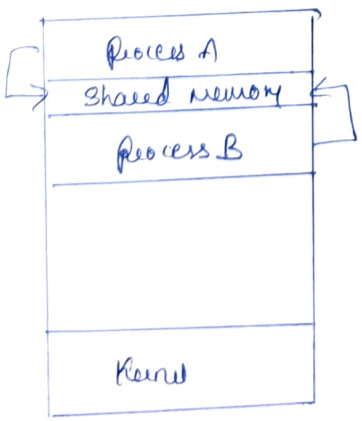
2 a) IPC is a mechanism provided by the OS that allows two or more process to communicate & exchange data with each other. 6M

The two approaches are:-

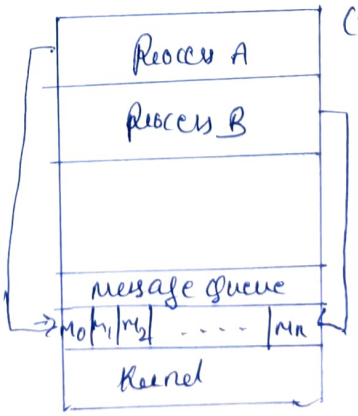
1) Shared-memory model
2) Message-passing model

1) Shared-memory :- Here two, or more processes communicate by accessing a common memory region created by the operating system. (1M)

2) Message-passing Model :- Here, processes communicate by exchanging messages without sharing a common memory region. The sending process sends a message to another process. The OS system manages message transmission. The receiving process receives the message. communication can be direct or indirect through mailboxes or message queues (1M)



Shared memory



message-passing (2M)

5

Q.No.	Answer description	Marks Allotted

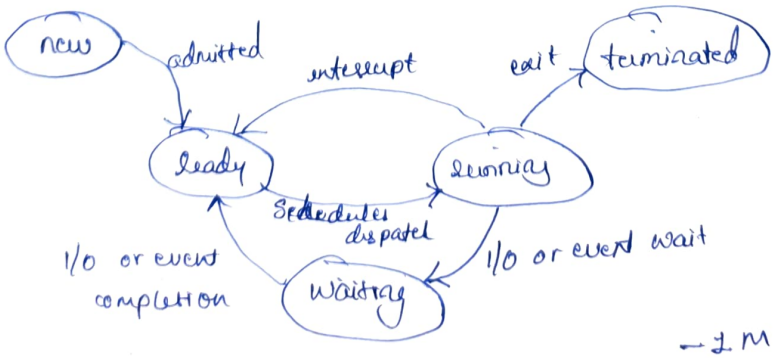
Comparison (2M)

Shared-memory	Message-passing
* Process communicate through a common memory region	* Process communicate by exchanging message
* Requires share memory to be established	* Does not require shared memory
* Generally faster for large data transfer	* Generally slower due to message-transfer overhead
Ex:- shmget() & shmat()	Ex:- msgsnd() & msgrcv()

b) Process concept: — 1M

* A Process is a Program in execution. It is an active entity that includes the program code, current activity, Program counter, CPU registers, stack, data section
* The OS manages process using a data structure called the Process control block (PCB)

6M



— 1M

Q.No.	Answer description	Marks Allotted

Process states are — 2M

1) New: The process is being created
2) Ready: The process is waiting to be assigned to the CPU
3) Running: Instructions are being executed
4) waiting: The process is waiting for some event to occur
5) Terminated: The process has finished execution.

— 2M

operations

1) Process creation: Creates a new process & state is moved from New → Ready.
2) Schedular dispatch: Assigns cpu to a ready process & starts running. State is moved from Ready → Running.
3) I/O Request: Process requests and I/O operation & state is moved from Running → waiting
4) I/O completion: I/O operation is completed & state is moved from waiting → Ready
5) Interrupt: CPU is taken away from the running process & state is moved from Running → Ready
6) Process Termination: Process completes execution & State is moved from running → terminated.

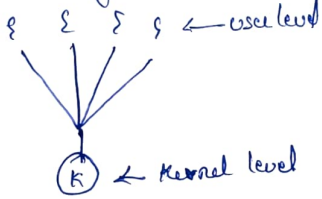
Q.No.	Answer description	Marks Allotted

Module - II

3) a) The three models are:-

1) Many-to-one model — 1M

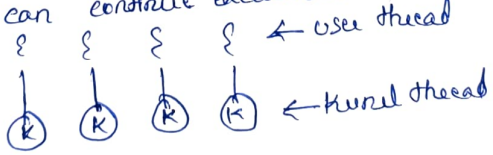
* Many user-level threads are mapped to a single Kernel thread.
* If one thread makes a blocking system call, the entire process gets blocked.



6M

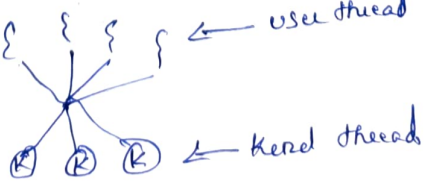
2) One-to-one model — 1M

* Each user-level thread is mapped to a separate Kernel thread.
* If one thread makes a blocking system call, other threads can continue execution



3) Many-to-many model — 1M

* Many user-level threads are mapped to an equal or smaller number of Kernel threads
* If one thread blocks, other threads can continue execution



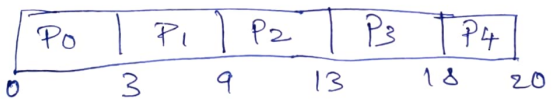
Q.No.	Answer description	Marks Allotted

Many-to-one	One-to-one	many-to-many
- many user threads mapped to one Kernel thread	One user thread mapped to one Kernel thread	many user threads mapped to many Kernel threads
- Thread creation overhead is low	thread creation overhead is high	thread creation overhead is moderate
- Blocking system calls blocks entire process	Blocking system calls blocks only the calling thread	Blocking system calls blocks only the associated kernel thread

– 3M

b) FCFS – 3M

Gantt chart



Waiting Time

$P_0 \rightarrow 0$

$P_1 \rightarrow 1$

$P_2 \rightarrow 5$

$P_3 \rightarrow 7$

$P_4 \rightarrow 10$ MS

Turn around time

$P_0 \rightarrow 3$

$P_1 \rightarrow 7$

$P_2 \rightarrow 9$

$P_3 \rightarrow 12$

$P_4 \rightarrow 12$

6M

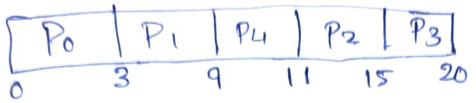
Average Waiting Time $= \frac{0+1+5+7+10}{5} = 4.6$ MS

Average Turnaround time $= \frac{3+7+9+12+12}{5} = 8.6$ MS

9

Q.No.	Answer description	Marks Allotted

SJF non Recemptive — 3M



Waiting time	Turn around time
$P_0 \rightarrow 0$	$P_0 \rightarrow 3$
$P_1 \rightarrow 1$	$P_1 \rightarrow 7$
$P_2 \rightarrow 7$	$P_2 \rightarrow 11$
$P_3 \rightarrow 9$	$P_3 \rightarrow 14$
$P_4 \rightarrow 1$	$P_4 \rightarrow 3$

Average Waiting Time = $\frac{0+1+7+9+1}{5}$ = 3.6ms

Average Turn around time = $\frac{3+7+11+14+3}{5}$ = 7.6MS

— or —

4 a) Benefits — 3M (any three)

i) Responsiveness :- Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a length operation, thereby increasing responsiveness to the user. 6M

Ex:- A multithreaded web browser could allow a user interaction in one thread, while an image was being loaded in another thread

10

Q.No.	Answer description	Marks Allotted

2) Resource Sharing :- By default, threads share code, data and other resources of the process to which they belong and allows multiple tasks to be performed simultaneously in a single address space

Ex: Multiple threads in a word processor can share document data while performing different operations.

3) Economy :- Allocating memory and resources for process creation is costly. Because threads share the resources of the process to which they belong, it is more economical to create and context-switch threads

Ex:- A web server uses multiple threads to handle client request instead of creating a separate process for every request.

Challenges — 3m (any 3)

1) Dividing activities :- This involves examining applications to find areas that ~~can~~ can be divided into separate, concurrent tasks, and thus run in parallel on individual cores.

Ex: Some steps in a sequential calculation ~~speed~~ depend on the results of previous steps.

2) Balance :- While identifying tasks that can run in parallel, programmer must also ensure that the tasks perform equal work of equal ~~work~~ value.

Q.No.	Answer description	Marks Allotted

Ex:- In parallel image processing, some threads may receive more image data to process than others.

3) Data dependency:- When one task depends on data from another, programmers must ensure that the execution of the tasks is synchronized to accomodate the data dependency.

Ex:- A thread attempting to calculate the total before another thread completes updating the shared data.

b) Handling of threading issues — 1M

fork()

* The fork() system call is used to create a separate, duplicate process
* When one thread in a program calls fork():
  - The new process can be a copy of the parent with all the threads.
  - The new process is a copy of the single thread only (that invoked the process)

Ex. pthread_create (&tid, attr, thread_function, NULL);
fork();

6M

exec() — 1M

* If a thread invokes the exec() system call, the program specified in the parameter to exec() will replace the entire process - including all threads

12

Q.No.	Answer description	Marks Allotted

* Which of the two versions of fork() to use depends on the application. If exec() is called immediately after forking, then duplicating all threads is unnecessary, as the program specified in the parameter to exec() will replace the process. In this instance, duplicating only the calling thread is appropriate. If however, the separate process does not call exec() after forking, the separate process should duplicate all threads

Signal handling - 2M

* A signal is used in UNIX system to notify a process that a particular event has occured.
* In multi-thread environment, the signal is delivered in variety of ways:
  - Deliver the signal to the thread to which the signal applies.
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process.
  - Assign a specific thread to receive all signals for the process.

Ex:- signal (SIGINT, handle_signal) → when the user presses Ctrl+C, the signal handler is executed.

Q.No.	Answer description	Marks Allotted

Thread cancellation – 1M

* Thread cancellation involves terminating a thread before it has completed.

Two types of thread cancellation:-

– Asynchronous cancellation:- one thread immediately terminates the target thread

– Deferred cancellation:- The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion

Ex:- pthread_cancel() → function for thread cancellation in pthreads

Thread-local storage – 1M

* Threads belonging to a process share the data of the process. However in some circumstances, each thread might need its own copy of certain data. We call such data a thread specific data or thread-local storage

* Ex:- In threads are used for transaction & each transaction has an ID. This unique ID is a specific data of the thread.

Q.No.	Answer description	Marks Allotted

Module III

5) a) A correct solution to the critical-section problem must satisfy the following three requirements

1) Mutual Exclusion – 2M

* If one process is executing in its critical section, no other process is allow to execute in its critical section.
* only one process can access the shared resource at a time.

6M

2) Progress – 2M

* If no process is executing in its critical section and some process wish to enter their critical sections, the selection of the next process must not be postponed indefinitely
* only processes that are ready to enter their critical sections should participate in the selection

3) Bounded waiting – 2M

* There must be a limit on the number of times other processes are allowed to enter their critical sections after a process has requested entry.
* This ensures that every waiting process gets an opportunity to execute.

15

Q.No.	Answer description	Marks Allotted
	- or -	
6 a)	The Structure of writer process — 2M do { wait (rw-mutex) .... /* writing is performed */ .... signal (rw-mutex); } while (true);	6M

	The Structure of reader process — 2M
	do {
	wait (mutex);
	read_count ++;
	if (read_count == 1)
	wait (rw-mutex);
	signal (mutex);
	....
	/* reading is performed */
	....
	wait (mutex);
	read_count --;
	if (read_count == 0)
	signal (rw-mutex);
	signal (mutex);
	} while (true);	

Q.No.	Answer description	Marks Allotted
	* The semaphores 'mutex' and 'rw_mutex' are initialized to 1; read_count is initialized to 0. * The 'mutex' semaphore is used to ensure mutual exclusion when the variable 'read count' is updated. * The 'read_count' variable keeps track of how many processes are currently reading the object. * The semaphor 'rw_mutex' functions as a mutual exclusion semaphore for the writers. * If a writer is in the critical section and 'n' readers are waiting, then one reader is queued on 'rw_mutex', and n-1 readers are queued on mutex. * When a writer executes signal (rw_mutex), we may resume the execution of either the waiting readers or a ~~sigh~~ single waiting writer. -2m	

Course Teacher

Domain Expert

Chairperson

17